TaskRabbit (UK) — Teardown & Build Brief

Prepared for building a similar-or-better on-demand home-services marketplace in the UK. Everything below is drawn from the live site (taskrabbit.co.uk), its rendered DOM, computed styles, and observed network traffic, plus clearly-labelled inference where something can't be seen directly from the browser.

---

1. What the product is

TaskRabbit is a **two-sided on-demand marketplace** that connects customers who need small home jobs done with vetted independent workers ("Taskers"). The customer searches a task, browses available Taskers by **price, skills, and reviews**, books a slot (as soon as same-day), then **chats, pays, tips, and reviews all inside the platform**. It is owned by IKEA, which is why furniture/IKEA assembly is the hero category.

The business model is a **service marketplace with a take-rate**: Taskers set hourly rates, the platform adds a service fee, and there's an insurance product ("TaskProtect", up to £1,000 cover). Trust is the core product: **ID-checked Taskers**, reviews, in-app payment, and 7-day support.

Headline trust/scale numbers shown on the homepage: 3.4M+ furniture items assembled, 1.5M+ moving tasks, 1M+ items mounted, 700k+ home repairs.

---

2. Service taxonomy (what the app offers)

The catalogue is organised as **8 top-level categories**, each with specific bookable tasks. This is the structure you'd replicate.

Category	**Example bookable tasks**	**Sample UK "starting at" price**
Assembly	Furniture Assembly, IKEA Assembly, Office Furniture, Bed, Wardrobe, Crib, SKYTTA Installation	Furniture £29 · IKEA £33

Category	Example bookable tasks	Sample UK "starting at" price
Mounting	Mount a TV, Mount Art or Shelves, Hang Pictures, Shelf Mounting, Light Installation	Mount Art/Shelves £35 · Mount TV £37
Moving	Moving Help, Heavy Lifting, Errand Running	—
Home Repairs	General home repairs, minor fixes	£700k+ done
Outdoor Help	Gardening, yard work	Gardening £30
Plumbing	Minor Plumbing Repairs	£54
Electrical	Electrical Help, Light Installation	£44
Cleaning	House Cleaning	—
(also surfaced)	Painting / Indoor Painting	£32

Design takeaway: the taxonomy is shallow and task-first — customers pick a *concrete job* ("Mount a TV"), not an abstract trade. Prices are shown as transparent "Projects starting at £X" anchors. Each category has its own landing page (SEO play), plus per-city pages ("Services By City") and "Cost Guides By Category".

---

3. Key pages & flows to replicate

- **Homepage** — hero search bar ("What do you need help with?"), category chips, popular projects with prices, testimonials, "How it works" (3 steps), trust badges, app-download CTAs.
- **Category / service landing pages** — one per task, price anchors, SEO copy, trending notes.
- **Services-by-city pages** — local SEO at scale.
- **Cost guide pages** — content marketing per category.
- **Search → Tasker results** — filter/sort by price, skills, reviews, availability.
- **Tasker profile** — reviews, rate, skills, ID-checked badge.
- **Booking flow** — pick Tasker → schedule → confirm.
- **In-app messaging + payment + tipping + reviews.**

- **Become a Tasker** (supply-side onboarding), **Elite Taskers**, **Help/Support**.
- **Trust:** TaskProtect insurance, ID checks, dedicated support.

How it works (the 3-step promise): 1) Choose a Tasker by price, skills, reviews → 2) Schedule as early as today → 3) Chat, pay, tip and review in one place.

---

4. Visual design system (measured from the live site)

Colour palette

All values below are the **actual computed colours** from the rendered page — safe to hand straight to a designer/dev as tokens.

Role	Hex	RGB	Where it's used
Primary green (CTA)	`#0D7A5F`	13, 122, 95	Search button, "See All Services", primary buttons
Brand green (headings)	`#2B4C32`	43, 76, 50	H1 / hero heading ("Find trusted help…")
Deep green (links)	`#0A2B14`	10, 43, 20	Text links
Ink / body text	`#242A30`	36, 42, 48	Paragraph + default text
Page background	`#F9FAFB`	249, 250, 251	Body — near-white, cool grey
Warm off-white	`#FAF9F5`	250, 249, 245	Secondary buttons / card fills
Accent blue	`#3860BE`	56, 96, 190	Secondary/utility buttons, IKEA tie-in
Footer grey	`#67727E`	103, 114, 126	Footer background
White	`#FFFFFF`	255, 255, 255	Cards, outlined chips, search field

Role	Hex	RGB	Where it's used
Logo mint *(visual, approx.)*	~`#3DBE8B`	—	The rabbit wordmark uses a brighter mint-green than the CTA green

Palette character: a **green-forward, trustworthy, "friendly-practical"** system. One dominant green family (mint logo → CTA teal-green → deep forest-green for type), a cool near-white ground, a warm off-white for softness, and a single blue accent. High contrast, calm, not flashy — signals reliability over excitement.

Typography

- **Single typeface: `Inter`** (with a system fallback stack: `Inter, "Inter Fallback", Helvetica, sans-serif`).
- Headings are large, bold, tight-tracked, and **green** (not black) — a signature move.
- Body is dark slate-grey `#242A30`, not pure black.

Iconography & imagery

- **Line/outline icons**, simple and rounded, one per category (assembly, mounting, moving, etc.).
- **Pill-shaped outlined chips** for quick service selection (white fill, thin border, rounded-full).
- **Shield/badge motifs** for trust (TaskProtect shield, ID-checked).
- Social icons in the footer: Facebook, X, Instagram, TikTok, LinkedIn (mono, in white on grey).
- App-store badges (Apple App Store + Google Play).
- Rounded corners throughout, generous whitespace, soft shadows on the search bar.

Component patterns to reuse

Rounded-full search bar with embedded button · outlined category pills · price-anchor cards ("starting at £X") · testimonial cards with reviewer name + service tag · numbered "how it works" row · trust-badge row · sticky header with hamburger on mobile.

---

5. Tech stack

Frontend — **directly observed**

- **Next.js (React)** — confirmed by `__NEXT_DATA__` and the `#__next` root element in the DOM. Server-rendered React, which is exactly right for an SEO-heavy marketplace (all those city/category pages).
- **Inter** webfont.
- Assets served from **AWS CloudFront** (`d34r8q7sht0t9k.cloudfront.net`, `assets.taskrabbit.com`) → strongly indicates **AWS** infrastructure.

Data, analytics & 3rd-party layer — **observed in network traffic**

- **Segment** (self-hosted `analytics.js`, served from `evs.sgp.taskrabbit.com/next-integrations/…`) — the customer-data / event pipeline.
- **Datadog RUM** (`browser-intake-datadoghq.com`) — real-user monitoring / observability.
- **Optimizely** — A/B testing & experimentation.
- **Microsoft Clarity** — session replay / heatmaps.
- **OneTrust** (`cdn.cookielaw.org`) — cookie consent / GDPR.
- **AppsFlyer** — mobile attribution.
- **Sprig** — in-product surveys.
- **Trustpilot** widget — reviews.
- **Zendesk** (`static.zdassets.com`) — support.
- Marketing pixels: Google Tag Manager, Facebook, TikTok, Pinterest, Reddit, Bing, Quantcast.

Backend, database & deployment — **inference (not visible from the browser)**

You cannot read someone's backend language or database from the front end, so treat this as informed inference, not fact:

- **Backend:** TaskRabbit is publicly known (from its engineering history) to run a **Ruby on Rails** monolith with services around it. The CloudFront/AWS footprint above is consistent with an **AWS-hosted** backend.
- **Database:** almost certainly a **relational core (PostgreSQL or MySQL)** — marketplaces are relational (users, taskers, bookings, payments, reviews) — likely with Redis for caching/sessions and Elasticsearch/OpenSearch for Tasker search.
- **Payments:** a PSP such as Stripe/Braintree (marketplace/Connect-style split payments) — not directly confirmed on the homepage.

- **Deployment:** AWS (CloudFront CDN + almost certainly containers/EC2 behind it).

When you brief Claude Code, present the backend as **"here's a modern equivalent stack"** rather than "copy theirs" — you can't verify theirs, and you can build something cleaner anyway (below).

---

6. Recommended stack to build similar — or better — in the UK

A pragmatic, modern, UK-compliant stack that matches or beats the original:

Frontend

- **Next.js (App Router) + React + TypeScript** — SSR/SSG for SEO on category & city pages (match TaskRabbit's strength).
- **Tailwind CSS** with the design tokens from §4 baked in.
- **Inter** (or a distinctive alternative if you want to differentiate the brand).
- Component lib: **shadcn/ui** or Radix for accessible primitives.

Backend

- Option A (fastest with Claude Code): **Node.js + NestJS (TypeScript)** or **Next.js API routes / tRPC** for a unified TS codebase.
- Option B (if you prefer Rails-like productivity): **Ruby on Rails 7** or **Django**.
- **REST or GraphQL** API; **WebSockets** for the in-app chat.

Database & search

- **PostgreSQL** (core relational data) + **Prisma** ORM if TypeScript.
- **Redis** for caching, sessions, rate-limiting.
- **Elasticsearch / OpenSearch** (or Postgres full-text to start) for Tasker search/filtering by price·skills·reviews·availability.

Payments & marketplace money-flow

- **Stripe Connect** — handles split payments, Tasker payouts, tips, and holds; UK/EU-ready, Strong Customer Authentication (SCA) compliant.

Infrastructure & deployment

- **Vercel** for the Next.js frontend (simplest), or all-in on **AWS** (ECS/Fargate + RDS Postgres + ElastiCache + CloudFront + S3) if you want one cloud.

- **Cloudflare** CDN/WAF as a cheaper CloudFront alternative.

Cross-cutting

- Auth: **Clerk / Auth0 / NextAuth**.
- Notifications: transactional email (Resend/Postmark) + push (Firebase) + SMS (Twilio).
- Observability: **Sentry** (errors) + **PostHog** (product analytics + session replay + A/B — replaces Segment+Optimizely+Clarity in one open tool).
- Consent/GDPR: **Cookiebot / Osano** (UK GDPR is mandatory).
- Reviews: build native, optionally surface **Trustpilot**.
- Maps/geo for local matching: **Mapbox** or Google Maps.

---

7. MVP scope + how to be *better* than TaskRabbit

MVP (two-sided marketplace core):

1. Service taxonomy + SEO landing pages (category, task, city, cost-guide).
2. Customer search → Tasker results with filter/sort (price, rating, availability, distance).
3. Tasker profiles (skills, reviews, rate, ID-verified badge).
4. Booking + scheduling (same-day slots).
5. In-app chat, payment (Stripe Connect), tipping, reviews.
6. Tasker onboarding + ID verification (e.g. Stripe Identity / Onfido).
7. Trust layer: insurance/guarantee, support.

Ways to beat it (differentiation ideas):

- **Voice-first booking** — describe the job by voice, get matched (a natural fit given your voice-engine ventures — reuse the shared voice stack).
- **Instant transparent quotes** — fixed-price up-front instead of "starting at", using a voice-to-quote flow.
- **Better local trust** — UK-specific: DBS checks, Gas Safe / NICEIC verification for plumbers/electricians.
- **Faster matching** — auto-match + instant accept, not manual browse.
- **Lower take-rate / clearer fees** as a wedge.
- **Sustainability angle** (TaskRabbit already teases "sustainable materials" trending).

---

8. Ready-to-paste briefs for Claude Code

8a. Frontend brief

Build a Next.js (App Router) + TypeScript + Tailwind marketplace frontend for an on-demand home-services platform (UK).

Design system (use as Tailwind tokens):

- Primary CTA green: #0D7A5F
- Heading green: #2B4C32
- Link green: #0A2B14
- Body text: #242A30
- Page bg: #F9FAFB
- Warm off-white (secondary buttons/cards): #FAF9F5
- Accent blue: #3860BE
- Footer grey: #67727E
- Font: Inter (bold, tight-tracked headings in green, slate-grey body)
- Style: rounded-full search bar with embedded button, outlined pill chips for categories, price-anchor cards ("Projects starting at £X"), testimonial cards, numbered 3-step "how it works" row, shield/badge trust icons, generous whitespace, soft shadows.

Pages:

- Home (hero search "What do you need help with?", 8 category chips, popular-projects price cards, testimonials, how-it-works, trust badges, app CTAs, footer).
- Category/task landing pages (SEO, price anchor, copy).

- City landing pages (programmatic SEO).

- Search results (filter/sort by price, rating, availability, distance).

- Tasker profile (reviews, skills, rate, ID-checked badge).

- Booking flow (choose tasker → schedule → confirm).

Make it fully responsive with a hamburger nav on mobile. SSR/SSG the SEO pages.

Categories & tasks: Assembly (Furniture, IKEA, Office, Bed, Wardrobe, Crib), Mounting (TV, Art/Shelves, Pictures, Lights), Moving (Moving Help, Heavy Lifting, Errands), Home Repairs, Outdoor/Gardening, Plumbing, Electrical, Cleaning, Painting.

8b. Backend brief

Build the backend for a two-sided home-services marketplace (UK).

Stack: NestJS + TypeScript (or Rails if preferred), PostgreSQL via Prisma, Redis cache, WebSockets for chat, Stripe Connect for split payments/payouts/tips.

Core domain models: User, Tasker (skills, hourly rate, availability, ID-verification status, service areas), Service/Task (category, task type, base price), Booking (status, schedule, line items, fees), Message/Thread, Review (rating, text, service tag), Payment/Payout, InsuranceClaim.

Endpoints/features:

- Auth (customer + tasker), ID verification hook (Stripe Identity/Onfido).

- Tasker search: filter/sort by price, rating, skills, availability, distance (Postgres full-text or OpenSearch; geo distance).

- Booking lifecycle: request → accept → scheduled → completed → reviewed, with same-day slots.

- Stripe Connect: authorise on booking, capture on completion, split platform fee, tips,

  tasker payouts.

- In-app chat (WebSocket), notifications (email/SMS/push).

- Reviews & ratings, trust/insurance ("TaskProtect"-style up-to-£1,000 cover) claim flow.

- UK compliance: GDPR data handling, SCA-compliant payments; optional DBS / Gas Safe /

  NICEIC verification fields on tasker profiles.

Deploy on AWS (ECS/Fargate + RDS Postgres + ElastiCache + CloudFront) or Vercel + managed

Postgres. Add Sentry + PostHog for monitoring and product analytics.

---

*Note on limits: frontend, design tokens, and the third-party/analytics layer are directly measured from the live site. Backend language, database engine, and deployment topology are **inferred** — they can't be read from a browser — so §5's backend and §6's recommendations are the "how you'd build an equal-or-better version" answer, not a claim about TaskRabbit's internals.*